AMS SCENARIO 3 — ENHANCEMENT · TECHNICAL BRIEF

How the target repository is selected

Three selection layers, three different mechanisms — and an honest account of which ones are measured.

MapleSure Insurance demo build · Verified against the codebase on 27 July 2026 · Companion to `S3_RANKING_VS_DUO.pdf`

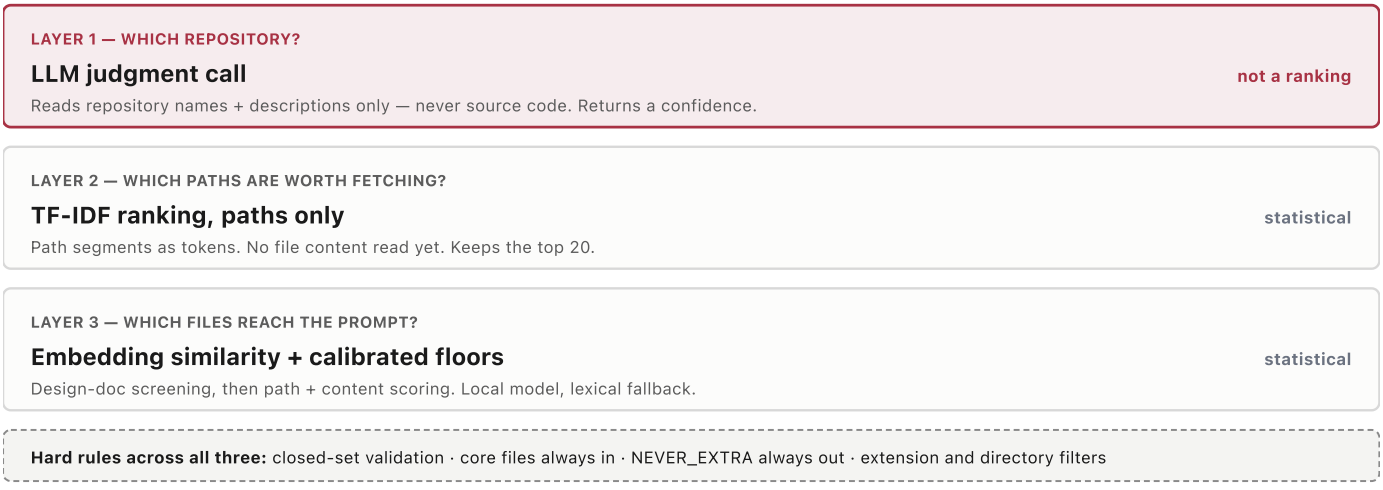
> THE ONE-LINE ANSWER
>
> *"Choosing the repository and choosing the files are different problems, so we solve them differently. The repository is a judgment call the model makes from names and descriptions — and it has to declare its confidence, because anything less than high stops and asks the developer. The files are picked by measured similarity scoring. Only the second one is a ranking."*

Three layers, three mechanisms

These get conflated constantly. They are not the same kind of decision and they do not fail the same way.

Figure 1 · The selection stack

Top to bottom, each layer narrows the problem for the one below it.



Only layers 2 and 3 are ranking. Layer 1 produces no scores at all — asking "what's the ranking accuracy of repo selection" is a category error, and the right answer is to explain the confidence gate instead.

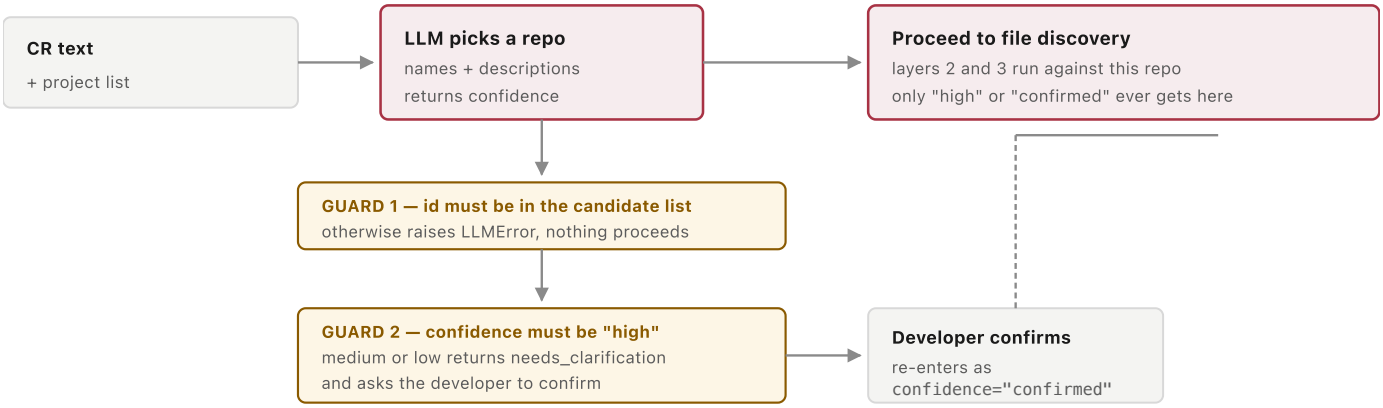
Layer 1 in detail — how the repository is chosen

`s3_enhancement/repo_match.py::suggest_target_repo` sends the CR text and a list of the connected GitLab projects to the model, and asks which one the change belongs to. The prompt is explicit that only names and descriptions are available: *"you have not read any source code."*

The model returns a best match id, a confidence of `high` / `medium` / `low`, one sentence of reasoning, and any genuinely plausible alternates.

Figure 2 · The decision path, including both refusals

Two guardrails wrap the judgment call. Neither is a score.



A wrong repo would poison everything downstream — file discovery, codegen and the diff would all be against the wrong codebase. That is why this layer stops and asks rather than proceeding on a guess. It is the same reflex as the CR-text clarity gate for vague tickets.

A confirmation request is recorded on the ticket timeline as `repo_match_confirmation_requested`, so the hesitation is auditable rather than a silent retry. Passing `confirmed_project_id` on the next call skips the model entirely and goes straight to scoping.

> **DO NOT DESCRIBE THIS AS DETERMINISTIC**
>
> This call deliberately omits a fixed cache key — its input genuinely varies (the CR text, and whoever's project list). It falls back to the default content-hash cache. Same CR and same project list gives the same answer; a reworded CR may not. That is correct behaviour here, but it means repo matching is **not** pinned the way the demo's narrative beats are.

How accurate is it? Split the question before answering

The layers have very different evidence behind them. Conflating them is how an honest system ends up sounding overclaimed.

Figure 3 · What is actually measured

Assurance by layer. Chips carry their own text — nothing here depends on colour.

LAYER	CORRECTNESS OF THE PICK	WHAT THE EVIDENCE ACTUALLY IS
1 · Repository LLM judgment	NOT MEASURED	Five tests in `tests/test_s3_repo_match.py`, all with a **stubbed model response**. They prove the plumbing is safe — malformed JSON raises, an out-of-list id raises, bad alternates are dropped, an empty candidate list raises. **None of them test whether the pick is right.** There is no labelled CR→repo set and no precision figure.
2 · Path shortlist TF-IDF	NOT MEASURED	Behaviour is tested (`test_s3_gitlab_relevance.py`), but there is no measurement of whether the true file survives into the top 20 on unseen repositories. Recall at this cut-off is unknown.
3 · File selection embeddings + floors	MEASURED	17 tests in `tests/test_s3_relevance.py`, plus a live check that every legacy subsystem stays screened out across three tier-name variants. Measured margins: nearest decoy subsystem 0.339 against a 0.45 floor; hardest real file 0.458 against a 0.55 floor. Selection is deterministic run to run.
Generation codegen + tests	MEASURED	`tools/verify_s3_live.py --gate N` runs N full live cycles; a cycle counts only if the live path genuinely ran (no silent replay) and the generated tests pass. Demo-day rule: **≥ 9 of 10**.

The measured layers are measured against one pinned CR per target. That is regression-proofing — it proves the pipeline has not drifted. It is not accuracy on unseen input, and it should never be presented as such.

> THE HONEST FORMULATION
>
> *"File selection we measure, and I can show you the margins. Repository selection we don't measure — we gate it. It has to declare high confidence or it stops and asks the developer, and that hesitation is recorded on the ticket. For a step where being wrong invalidates everything downstream, we chose a human check over a number we'd have to defend."*

Why measuring layer 1 is genuinely hard

- **No ground truth exists yet.** Accuracy needs a labelled set of real CRs mapped to their correct repository. Nobody has built one, and it would have to come from the client's own history to mean anything.
- **It depends on someone else's metadata.** The model sees only names and descriptions. On an estate where repository descriptions are blank or stale, no model does well — and that quality is not ours to control.
- **The right metric is not just top-1.** Because the gate keys on confidence, what matters as much is *calibration*: when it says "high", how often is it right? A well-calibrated 80% is safer here than a poorly-calibrated 90%.

What a real evaluation would look like

If the client asks for a number, this is the work that would produce a defensible one — worth quoting as a scoped follow-on rather than improvised on the spot.

STEP	WHAT IT INVOLVES
Build a labelled set	50–100 historical CRs from the client's own tracker, each tagged with the repository it was actually delivered against. This is the expensive part and the only part that cannot be skipped.
Measure top-1 accuracy	How often the best match is the correct repository, reported separately for repos with good descriptions and repos without — the split will be large and is itself the useful finding.
Measure calibration	Accuracy within each confidence bucket. The gate is only sound if "high" really is more reliable than "medium". If it isn't, the threshold moves.
Measure the escalation rate	What share of CRs stop to ask the developer. Too high and the automation is not saving anyone time; too low and the gate is not protecting anything.
Measure layer-2 recall	How often the file that actually needed changing survives into the top 20 path shortlist. A miss here is silent, which makes it the most valuable number on this list.

> **THE SILENT-FAILURE POINT, WORTH MAKING UNPROMPTED**
>
> Across every layer, the dangerous error is exclusion, not inclusion. Something wrongly included costs tokens and is visible in the file panel. Something wrongly *excluded* is never opened, so no downstream check can notice it was missing — `verify_core_recall()` only validates the declared core files of the target already chosen. Recall, not precision, is where the measurement effort belongs.

Method. Every claim here was read from the codebase on 27 July 2026: `s3_enhancement/repo_match.py`, `api/routers/s3.py`'s `/gitlab/scope-auto`, `s3_enhancement/relevance.py`, `tests/test_s3_repo_match.py`, `tests/test_s3_relevance.py` and `tools/verify_s3_live.py`. Test counts and threshold margins were taken from live runs, not from code comments — several comments have drifted from the values the code now produces. See `docs/S3_RANKING_VS_DUO.pdf` for the file-selection funnel in detail, and `docs/design/design_doc_feedback_loop.md` for a related open gap in how design docs age.